Cohesion.agda — the three cohesion modalities

Where this file sits

`agda/ModalHoTT/Modalities/Cohesion.agda` is the floor of the whole synthetic (axiomatic) track. Everything downstream — the adjunctions, the properties files, the fracture theorem, the de Rham work the project is currently pointed at — ultimately leans on the three modalities introduced here. If you only ever read one file from `agda/`, this is the one that buys you the most context per line.

A few things to hold in mind before we go in.

HoTT, briefly. Homotopy Type Theory ("HoTT") is the flavor of type theory the project lives in. The one mental swap to make up front: a type, in HoTT, is not a set of elements — it's a *space*. A space has points (the elements), but it also has paths between points (these are equalities, in HoTT's reading), paths between paths, and so on. This isn't decoration; it's the whole point. Equality is interesting in HoTT in a way it isn't in your usual programming language, because two things being "equal" carries data: *which path* witnesses the equality matters.

> **References**
>
> - The HoTT Book — Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*, 2013
> - nLab: homotopy type theory

What a modality is. A modality, in this setting, is a systematic operation that takes a type and gives you back a "filtered" version of that type — a version seen through a particular lens. The project uses three:

- *flat* (♭), which keeps only the discrete structure of a type — its set-of-points reading, with any connective tissue thrown away.
- *sharp* (♯), which makes a type maximally connected — codiscrete, every point glued to every other.
- *shape* (Σ), which keeps the homotopy skeleton (paths between points, paths between paths, …) and forgets the underlying geometry.

You'll meet each in turn below. The fact that all three exist on the same type, and play off each other in specific ways, is what makes the setting *cohesive*.

> **References — modalities in HoTT**
>
> - Rijke–Shulman–Spitters, *Modalities in homotopy type theory*, 2020 (arXiv)
> - nLab: modality (in homotopy type theory)
> - nLab: flat modality
> - nLab: sharp modality
> - nLab: shape modality
>
> **References — the cohesive setting**
>
> - Lawvere, *Axiomatic cohesion*, 2007 (nLab entry)
> - Shulman, *Brouwer's fixed-point theorem in real-cohesive HoTT*, 2018 (arXiv)

- Schreiber, *Differential cohomology in a cohesive ∞-topos* (arXiv)
- nLab: cohesive (∞,1)-topos
- nLab: real-cohesive homotopy type theory

What the file actually does. It postulates these three modalities. *Postulates*, not constructs. The project is openly upfront about this: in a more refined type theory (real-cohesive HoTT, à la Shulman), most of what's postulated here would be either derivable or built into the type theory's rules. Agda doesn't have those rules. So we declare what we want with the type signatures we'd expect, and trust the rest of the math to behave. The analytic track (`agda/Analytic/`) exists in parallel precisely to check that the postulates aren't vacuous — that they actually distinguish what they're supposed to distinguish.

The semantic intent (which never appears as an Agda object in this file) is that all this is happening on the site B$\mathbb{Z}$ — the delooping of the integers, which you can read as the one-object groupoid built from the group $\mathbb{Z}$. You don't need to fully unpack that yet. Just know there's a specific intended model behind the axioms, and that's what justifies which axioms we postulate and which we don't.

References

- nLab: delooping
- nLab: classifying space

OK. Into the file.

Chunk 1 — pragma and module header

```
{-# OPTIONS --cubical --guardedness #-}

module ModalHoTT.Modalities.Cohesion where
```

The `{-# OPTIONS ... #-}` line is a *pragma*: an instruction to Agda about how to compile this file. `--cubical` puts us in cubical Agda — a flavor where paths and equivalences are first-class, computational objects rather than logical fictions. (Cubical is what makes HoTT actually run, computationally. Without it you get HoTT's spirit but not its mechanics.) `--guardedness` enables coinductive types; we don't use them in this file, but the cubical standard library is built with this flag on, so every file in the project mirrors it.

References

- Cohen–Coquand–Huber–Mörtberg, *Cubical Type Theory: a constructive interpretation of the univalence axiom*, 2018 (arXiv)
- nLab: cubical type theory

`module ModalHoTT.Modalities.Cohesion where` declares the file's identity. Agda's module system maps onto folders: the dots are directories, so this is the file at `agda/ModalHoTT/Modalities/Cohesion.agda`. Other files in the project import this module by its full name.

Chunk 2 — the file's own preamble comment

```
-- Cohesion modalities for modal HoTT, postulated as axioms.
--
-- Semantic background: the cohesive ∞-topos is modelled by the site B𝕫
-- (delooping of 𝕫 ≃ S¹ as ∞-groupoids, but foregrounding the algebraic/
-- covering structure of 𝕫). The site is not an Agda type; the modalities
-- are axiomatic, justified by the B𝕫 semantics.
--
-- Adjunctions (♭ ⊣ ♯, ∫ ⊣ ♭) are stubbed; see Adjunctions.agda.
```

Worth reading slowly — it's doing real work.

"Postulated as axioms" is the strong claim. We're not constructing anything in this file; we're declaring axioms by writing type signatures with no implementations.

"The site is not an Agda type" is the honest disclaimer. The intended *meaning* of everything we postulate is grounded in B𝕫, but B𝕫 itself never shows up in the code. We're writing in the language of cohesive types, and B𝕫 is the model that justifies the axioms. Think of it as the file's silent partner. (The comment notes that B𝕫 ≃ S¹ as ∞-groupoids — they're the same homotopy type — but the project deliberately foregrounds the algebraic side of 𝕫 rather than the topological side of S¹. That's a project-wide decision; see `workflow/status.md` and `workflow/only_read_if_asked_to/conversation.md` if you want the reasoning.)

The last line previews the bigger picture: the three modalities aren't independent. They sit in two adjunctions — `♭ ⊣ ♯` and `∫ ⊣ ♭`, where `⊣` is the standard category-theory notation for "is left adjoint to." That structure is what makes them *cohesive* rather than three separate operations. This file sets the modalities up; `Adjunctions.agda` ties them together.

Chunk 3 — imports and universe variables

```
open import Cubical.Foundations.Prelude
open import Cubical.Foundations.Equiv
open import Cubical.Foundations.Function

private
  variable
    ℓ ℓ' : Level
```

Three imports from the cubical Agda library. `Prelude` gives us the path type and the basics. `Equiv` gives us equivalences (we'll use `isEquiv` shortly). `Function` gives composition and a few other plumbing utilities.

The `private variable` block is universe polymorphism. In Agda, `Type` isn't one giant collection of all types; there's an infinite tower `Type₀, Type₁, Type₂, …` (the indexing dodges Russell-style paradoxes — `Type` doesn't sit inside itself, but inside the next level up). Anything we want to apply to types of arbitrary level needs to be parameterized by a level. Declaring `ℓ ℓ' : Level` here lets us use them freely in any definition below, and Agda will silently quantify them when needed. It's pure ergonomics — without it we'd have to write `∀ {ℓ} → …` on every signature.

Chunk 4 — flat modality: the operator and the counit

A brief orienting note before we step into the flat block. None of what follows is built. Each `postulate` is a label affixed to the face of a piece of equipment we don't actually unbox — we name the controls the device has and what they do, while the wiring behind them lives in the intended B$\mathbb{Z}$ model. The metaphor will sharpen at chunk 10.

```
postulate
  FlatModality : Type ℓ → Type ℓ

  -- ε : ♭A → A
  flat-counit : {A : Type ℓ} → FlatModality A → A
```

We open the flat modality (♭). First the operator itself: `FlatModality` takes a type and gives back a type — its "flat version," `♭A`. Semantically: the same underlying points, but with the discrete topology forced on them (any connective tissue thrown away).

Then `flat-counit`. The comment names it ε (epsilon), the standard symbol for the counit of a comonad — and flat is a comonad. (If "comonad" sounds heavy: it just means we have an operation `♭` plus a way to project back out of it, with some coherence conditions. Like a monad with the arrows flipped.) Read the signature: given a `♭A`, produce an `A`. So we can always "forget the discreteness" and recover the underlying type.

The curly braces around `{A : Type ℓ}` mark `A` as an *implicit argument*. We don't pass it at call sites; Agda infers it from context. It's a little like a default parameter in Ruby/JS, but for types, and inferred rather than defaulted.

`postulate` is the keyword that makes this all axiomatic: we're stating what exists, not building it. Agda will trust these declarations as if they were primitive.

Chunk 5 — flat introduction, and a first honest disclosure

```
  -- [−] : A → ♭A  (crisp introduction; in real-cohesive HoTT this is only
  -- valid when the input is in a crisp context; postulated unconditionally
  -- here because Agda lacks native crisp-context support)
  flat-intro : {A : Type ℓ} → A → FlatModality A
```

The other direction: given `a : A`, produce a `♭a : ♭A`. The notation often used for this is `[a]` (square brackets), hence the comment's `[−]`.

Now the comment is doing major work, and we should not gloss it. In *real-cohesive HoTT* — the more disciplined theory we're loosely tracking — `flat-intro` is not allowed for arbitrary `a`. It's only allowed when `a` lives in a "crisp context," meaning the variable doesn't depend on any continuously varying data. The restriction is what makes the whole flat story coherent; without it, the discrete reflection isn't actually discrete in any meaningful sense.

Agda has no notion of crisp contexts. So we postulate `flat-intro` unconditionally, knowing this is an over-approximation. **This is a known shortcut.** It is flagged in `workflow/only_read_if_asked_to/rug.md`. The project does not pretend otherwise. The reason it's tolerable in practice is that downstream theorems still come out right *if* you imagine the inputs are always crisp, and the analytic track provides separate sanity checks that we're not relying on anything load-bearing that the shortcut would break.

> **References**
>
> - Shulman, *Brouwer's fixed-point theorem in real-cohesive HoTT*, 2018 (arXiv) — §2 for the crisp formal calculus
> - nLab: flat modality — "Modal type theory" section covers crisp introduction

This is, by the way, one of the most interesting kinds of postulate in the file: the kind where we're knowingly stretching beyond what the meta-theory permits, in exchange for being able to work at all. Recognizing these is half the skill of reading axiomatic Agda code.

Chunk 6 — flat induction

```
-- Comonadic induction: eliminate ♭A into a dependent type over ♭A.
flat-ind : {A : Type ℓ} (B : FlatModality A → Type ℓ')
           → ((a : A) → B (flat-intro a))
           → (u : FlatModality A) → B u
```

This is the technically chunkiest postulate in the file. Slow down.

A *dependent type* `B : ♭A → Type ℓ'` is a family of types indexed by `♭A`. For each `u : ♭A`, there's a corresponding type `B u`. (Non-dependent example for intuition: imagine a function that, given a user, returns the type of that user's settings — a different type per user.) Dependent types are how type theory talks about predicates and propositions: a predicate "P of x" is just a family of types indexed by x, where "P of x is true" means "the type P x has an inhabitant."

Now `flat-ind` reads: if for every `a : A` I can produce a value of type `B (flat-intro a)`, then I can produce a value of type `B u` for every `u : ♭A`.

In English: *to define a function on `♭A`, it's enough to define it on inputs of the form `flat-intro a`*. This is the induction principle for the flat modality — its version of "case analysis." It's the part that gives flat its real teeth: it says `♭A` has no extra inhabitants beyond what `flat-intro` produces (up to coherent paths).

A rough Ruby/JS analogy: if you have a sealed class with one constructor case, and you want to define a method whose return type may depend on the case, you only have to handle that one case. `flat-ind` says `♭A` is in this sense "sealed by `flat-intro`."

Chunk 7 — the beta rule for flat-ind

```
flat-ind-β : {A : Type ℓ} (B : FlatModality A → Type ℓ')
             (f : (a : A) → B (flat-intro a)) (a : A)
             → flat-ind B f (flat-intro a) ≡ f a
```

The *computation rule* (β-rule, "beta rule") for `flat-ind`. If you define a function on `♭A` by handing `flat-ind` a function `f` over `A`, and then apply your defined function to a freshly-built `flat-intro a`, you should get back `f a`. That is, the induction principle "does the obvious thing" on `flat-intro` inputs.

A short word about `≡`. In cubical Agda, `x ≡ y` is the *path type* between `x` and `y`. An element of `x ≡ y` is a path — a continuous deformation from `x` to `y`. If such a path exists, we say `x` and `y` are propositionally equal. The reason this matters: in HoTT, equality isn't a yes/no proposition, it's itself a type, and the elements of that type — the paths — can be compared, composed, and reasoned about. For now you can read `x ≡ y` as "x equals y," but keep in the back of your mind that there's more structure there.

Chunk 8 — flat-counit-intro, the obvious roundtrip

```
-- In real-cohesive HoTT this holds definitionally (crisp η-rule for ♭).
flat-counit-intro : {A : Type ℓ} (a : A) → flat-counit (flat-intro a) ≡ a
```

If I take an `a`, wrap it as a `♭a`, and then forget the structure to land back in `A`, I should get `a`. Of course.

The comment is, again, an honesty disclosure. In real-cohesive HoTT this is *definitionally* true: the type checker sees `flat-counit (flat-intro a)` and `a` as the same expression — no proof required, no path to construct. In our Agda, definitional equality doesn't hold here, so we postulate the propositional version: there *is* a path between the two, but the type checker won't reduce one to the other on its own. Minor cost, structurally innocent.

Chunk 9 — when is a type already flat?

```
IsFlatModal : Type ℓ → Type ℓ
IsFlatModal A = isEquiv (flat-counit {A = A})
```

Notice: no `postulate` here. This is a *definition*. We're defining what it means to be flat-modal.

A type `A` is flat-modal when the counit `flat-counit : ♭A → A` is an equivalence — when applying the modality and then forgetting it gives back the same type. Operationally: `A` already is its own flat reflection. There's nothing extra to forget because there's nothing beyond the discrete structure to begin with. Examples in spirit: the integers ℤ, a finite set, any "purely set-like" type.

`isEquiv` is from the cubical library. In HoTT, an equivalence is more subtle than a bijection — you need not just an inverse but coherence data showing the inverse plays well with paths. The library packages that for us, and for now you can read `isEquiv f` as "f has an inverse, with all the right coherences."

A forward pointer worth giving here: `IsFlatModal` becomes the workhorse type in the downstream files. The big *negative* result of the analytic track is that `B𝕫` (our intended site) is *not* flat-modal — the counit at `B𝕫` is not an equivalence — which is exactly what you'd want, since `B𝕫` is supposed to be a non-discrete space. That negative result is the strongest claim the analytic track can make without crisp-context infrastructure. (See `agda/Analytic/ModalHoTT/Modalities/Cohesion.agda` if you want to chase it.)

That closes the flat block. Across chunks 4–9 we declared a fixed set of operations — operator, counit, intro, induction, β-rule, η-rule on the counit, plus the question `IsFlatModal` — without constructing any of them. Shape and sharp follow the same template, and starting at chunk 10 we'll have a sharper picture for how to see it.

Chunk 10 — shape modality (Σ): installing the shape console

```
postulate
  ShapeModality : Type ℓ → Type ℓ

  -- η : A → ∫A
  shape-unit : {A : Type ℓ} → A → ShapeModality A

IsShapeModal : Type ℓ → Type ℓ
IsShapeModal A = isEquiv (shape-unit {A = A})

postulate
  -- Non-dependent recursion: map out of ∫A into any shape-modal type.
  -- Dependent induction (for shape-modal fibrations) is deferred.
  shape-rec : {A : Type ℓ} {B : Type ℓ'}
              → IsShapeModal B
              → (A → B)
              → ShapeModality A → B

  shape-rec-β : {A : Type ℓ} {B : Type ℓ'}
                (Bm : IsShapeModal B) (f : A → B) (a : A)
                → shape-rec Bm f (shape-unit a) ≡ f a
```

A short shift in mental picture before we go in. Picture the front face of a piece of rack-mounted lab or studio equipment: a panel covered in labeled jacks, indicator lamps, calibration LEDs, dials, and the occasional unsoldered slot where a feature wasn't installed. Everything the device can do is something you can point at on the face of it; the actual wiring lives *behind* the panel and is not what you describe when you describe the console. That's exactly our situation here — what we describe in Agda are the labeled controls on the face; the wiring behind them is the intended semantic model on B𝕫, which doesn't appear in the file.

The shape modality (Σ) is one such console — or more precisely, a *family* of consoles, one per input type A. This block postulates five parts that come with every one of them. None of the parts is built; each is declared, by labeling a socket on the panel without anything wired behind it. We'll go through them in dependency order.

1. The declaration of the family. `ShapeModality : Type ℓ → Type ℓ`. This says: every type A has a shape-console, called `∫A`. We're not constructing these consoles; we're declaring that one exists for each

input type.

2. The entry jack. `shape-unit : A → ∫A`. A socket on the console's face labeled "input." You feed in a raw `a : A`; out the back (semantically) comes `shape-unit a : ∫A`. The comment names this η — the symbol for the unit of a monad — and that flags something important. Flat had a *counit*, an arrow `♭A → A` pointing *out of* the modality. Shape has a *unit*, pointing *into* it. That's the difference between a comonad (flat) and a monad (shape). Same family of structure, opposite direction of flow.

3. An indicator lamp. `IsShapeModal A`. This is the only line in the block that isn't a `postulate` — it's a real definition, built from the postulates around it. `IsShapeModal A` is the type whose inhabitants are proofs that `shape-unit` at A is an equivalence — that the entry jack on A's console is invertible. If the lamp is lit, A is *already* its own shape; pumping it through the modality changes nothing. (For the cohesive setting, the lamp lights on types with "no geometry to lose" — homotopy-pure types.)

4. The output jack, with a precondition. `shape-rec`. To pull a value out of ΣA into a target type B, the console demands two things, both wired in before it'll open the output jack: first, *the lamp on B's console must be lit* (`IsShapeModal B` — the target must itself be a shape-pure type), and second, *a function* `A → B` saying what to do with the raw inputs. Hand the console both, and it returns a function `∫A → B`. The precondition is the whole game: without it, anyone could pull a `∫A` back into any old type and the modality would be transparent. The precondition is what makes Σ a *restriction* on where you can exit to from the shape-side — you can only land in types that are themselves already shape-pure.

5. A calibration LED. `shape-rec-β`. A guarantee about how the entry and exit jacks compose. Take a raw `a : A`, run it through the entry jack to get `shape-unit a`, then run *that* through the output jack with handler `f` — and you get back exactly `f a`. The roundtrip "enter, then exit-with-this-handler" is the identity on raw inputs. Without this rule, the entry and exit jacks could be wired to different things behind the panel and we wouldn't notice; with it, the console is calibrated to itself.

An unsoldered slot. Flat had `flat-ind`, an exit rule that lets you map `♭A` into a *family* of types varying over `♭A`. The shape console doesn't ship with the corresponding `shape-ind`. The comment in the file names the reason: a dependent eliminator would require fiberwise modality conditions — at every fiber of the family, the type sitting there has to itself be shape-modal — and the project hasn't pinned those down. It's tracked as an open question in `workflow/status.md`. The consequence ripples: several maps in `Properties.agda` that *would* be theorems if `shape-ind` were installed are stuck as postulates instead. Any `shape-` postulate downstream with a comment about "blocked on dependent induction" originates at this empty slot.

Chunk 11 — sharp modality (♯): installing the sharp console

```
postulate
  SharpModality : Type ℓ → Type ℓ

  -- η : A → ♯A
  sharp-unit : {A : Type ℓ} → A → SharpModality A

IsSharpModal : Type ℓ → Type ℓ
IsSharpModal A = isEquiv (sharp-unit {A = A})
```

```
postulate
  sharp-rec : {A : Type ℓ} {B : Type ℓ'}
              → IsSharpModal B
              → (A → B)
              → SharpModality A → B

  sharp-rec-β : {A : Type ℓ} {B : Type ℓ'}
                (Bm : IsSharpModal B) (f : A → B) (a : A)
                → sharp-rec Bm f (sharp-unit a) ≡ f a
```

The sharp modality (♯). Structurally a near-mirror of the shape console: same five parts in the same dependency order — family declaration, entry jack, indicator lamp, output jack with precondition, calibration LED. Same unsoldered slot where the dependent eliminator would be (`sharp-ind` is missing for the same reason `shape-ind` is, with the same downstream consequence). What changes between the two consoles is what each panel *means*, not how it's wired on the face.

Shape projects onto the homotopy skeleton — keep the paths between points, forget the geometry. Sharp does something different: it makes the type maximally connected, "codiscrete." Every point becomes path-connected to every other point. If shape forgets geometry by *keeping the skeleton*, sharp forgets local distinctions by *gluing everything together*. Same console layout, different intended semantics behind the panel.

The honest current state of the sharp leg: in this project, it's the weakest of the three. Synthetically (this file) the postulates are no harder than for shape. Analytically (the parallel file `agda/Analytic/ModalHoTT/Modalities/Cohesion.agda`) the sharp console is *deferred entirely* — there is no concrete cubical Agda construction known for `♯(B𝕫)` that's compatible with the rest of the cohesive picture. `workflow/status.md` has a section "Unlocking ♯ — what it costs and what it buys" laying out what would change if the sharp leg got built out and what stories it would open: differential cohomology, the Poincaré lemma, the de Rham separation result the project is currently pointed at. If you're reading this file because you're chasing the de Rham arc, the underdevelopment of sharp is the rate-limiting step downstream.

Chunk 12 — adjunction stubs (closing comment)

```
-- ------------------------------------------------------------------------
-- Adjunction stubs
--
-- These tie the three modalities into a genuine cohesive structure.
-- Deferred to Adjunctions.agda.
--
-- ♭ ⊣ ♯ : (FlatModality A → B) ≃ (A → SharpModality B)
-- ∫ ⊣ ♭ : (ShapeModality A → B) ≃ (A → B)   [for B flat-modal]
-- ------------------------------------------------------------------------
```

A closing note rather than code. Read it as the bridge to the next file.

So far we have three modalities, each with its own internal structure — three consoles, each in its own rack slot, none yet patched to the others. What makes them *cohesive* — what makes them a single coherent setting rather than three independent operations — is the way they get wired together: the adjunctions `♭ ⊣ ♯` and `∫ ⊣ ♭`. Each `⊣` is a *patch cable* between two consoles: it links a jack on one to a jack on another by a natural bijection of hom-sets. For example, `♭ ⊣ ♯` says maps out of `♭A` into `B` correspond to maps from `A` into `♯B` — the cable runs between the exit-side of flat and the entry-side of sharp, up to a natural bijection (the `≃` is that bijection, an equivalence in HoTT's sharper sense).

`Adjunctions.agda` runs those cables. With them in place, the three consoles aren't independent any more — they form one rack, and that rack is what we mean by *cohesive*. From there the project develops to `Properties.agda`, `Fracture.agda`, and into the de Rham material the current task is pointed at, all of which are, in one form or another, computing with the three patched-together consoles.

> **References**
>
> - Schreiber, *Differential cohomology in a cohesive ∞-topos*, §3 (arXiv) — the adjoint-modality perspective on the cohesive triple `∫ ⊣ ♭ ⊣ ♯`
> - nLab: adjoint modality